04

Funções

Apostila do aluno · duração da aula: 2h30

O que você vai aprender

- 1. O que é uma função?
- 2. Parâmetros e argumentos
- 3. return: a função devolve um resultado
- 4. Valor padrão
- 5. Arrow functions
- 6. Escopo: onde a variável existe
- 7. Funções com if e laços

Antes de começar

Esta aula usa tudo o que vimos até aqui: condicionais, arrays e laços. E finalmente vamos entender o `(fruta) => { }` do `forEach` da Aula 3. Todos os exemplos e exercícios podem ser executados direto no navegador, sem instalar nada, em **site-alunos-nu.vercel.app**.

Prof. Walter Caciano · Greenn School

1 O que é uma função?

Uma função é um **bloco de código com nome**, que você escreve **uma vez** e usa **quantas vezes quiser**. Pense numa **receita de bolo**: você escreve a receita uma vez (**declara**) e, toda vez que quiser um bolo, segue a receita (**chama**).

```
// 1. Declarar: escrever a receita
function saudar() {
  console.log("Olá, turma!");
}

// 2. Chamar: usar a receita
saudar();
saudar();
```

> **Atenção:** declarar não executa nada. O código só roda quando você **chama** a função, com os parênteses: `saudar()`.

Você já usa funções prontas desde a primeira aula: `console.log()`, `Math.floor()`, `frutas.push()`.

2 Parâmetros e argumentos

Parâmetros são as **informações que a função recebe** para trabalhar. Funcionam como variáveis que só existem dentro dela.

```
function saudar(nome) {          // nome = parâmetro
  console.log(`Olá, ${nome}!`);
}

saudar("Ana");                   // "Ana" = argumento
saudar("Caio");

function apresentar(nome, idade) {
  console.log(`${nome} tem ${idade} anos`);
}
apresentar("Bia", 20);           // a ordem importa!
```

Termo	Onde aparece	No exemplo
Parâmetro	na **declaração**, entre parênteses	`nome`
Argumento	na **chamada**, o valor que você manda	`"Ana"`

3 `return` : a função devolve um resultado

Com **return**, a função **devolve** um valor para quem chamou. Esse valor pode ser guardado numa variável, usado numa conta ou mostrado depois.

```
function somar(a, b) {
  return a + b;
}

const resultado = somar(3, 5);   // resultado = 8
console.log(resultado);          // 8
console.log(somar(10, 20) * 2);  // 60
```

return x console.log — a dúvida mais comum

	console.log	return
O que faz	**mostra** na tela	**devolve** o valor para quem chamou
Guardar numa variável?	não (vira `undefined`)	sim
Usar numa conta?	não	sim

```
function dobroMostra(n) { console.log(n * 2); }
function dobroDevolve(n) { return n * 2; }

const a = dobroMostra(5);    // mostra 10, mas a = undefined
const b = dobroDevolve(5);   // não mostra nada, mas b = 10
console.log(b + 1);          // 11
```

> **Regra prática:** a função calcula e faz `return`; quem chama decide se mostra com `console.log(minhaFuncao())`. E lembre: o `return` **encerra a função** na hora.

4 Valor padrão

Se o argumento não for enviado, o parâmetro usa o valor padrão definido com `=`:

```
function aplicarDesconto(valor, porcentagem = 10) {
  return valor - valor * porcentagem / 100;
}
aplicarDesconto(100);       // 90  (usou 10%)
aplicarDesconto(100, 25);   // 75
```

5 Arrow functions

Outra forma de escrever função, mais curta, guardada numa variável. Com uma linha só, o `return` fica automático:

```
function dobro(n) {          // forma tradicional
  return n * 2;
}

const dobro2 = (n) => {      // arrow function
  return n * 2;
};

const dobro3 = (n) => n * 2; // arrow curta: return automático
```

> **Lembra do forEach da Aula 3?** Aquele `(fruta) => { }` é uma arrow function! A gente entrega uma função para o forEach, e ele chama essa função uma vez para cada item: `frutas.forEach(mostrar)`.

6 Escopo: onde a variável existe

Uma variável criada **dentro** da função só existe **dentro** dela:

```
function calcular() {
  const segredo = 42;
  console.log(segredo);     // 42
}
calcular();
console.log(segredo);       // ERRO: segredo is not defined
```

Uma variável criada fora pode ser lida dentro da função, mas prefira passar valores por **parâmetro**: fica mais claro de onde vem cada coisa.

7 Funções com `if` e laços

Dentro de uma função cabe tudo o que já aprendemos. **Boas práticas:** dê nomes com verbos (`calcularMedia`, `ehPar`) e faça cada função cuidar de uma tarefa só.

```
function situacao(nota) {
  if (nota >= 7) return "Aprovado";
  if (nota >= 5) return "Recuperação";
  return "Reprovado";
}

function somarLista(lista) {
  let total = 0;
  for (const n of lista) total += n;
  return total;
}

console.log(situacao(6));               // Recuperação
console.log(somarLista([1, 2, 3, 4]));  // 10
```

8 Exercícios

Do 1 ao 4 você cria e chama funções; do 5 ao 8 usa `return` com `if`; do 9 ao 12 junta valor padrão, arrow function e laços. Atenção: quando a função usa `return`, mostre o resultado com `console.log(...)`. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 Saudação — NÍVEL 1

Crie `saudar(nome)` que mostra "Olá, nome!" e chame com `saudar("Ana")`.

Saída esperada: Olá, Ana!

02 Dobro — NÍVEL 1

Crie `dobro(n)` que **devolve** (return) o dobro de n. Mostre `console.log(dobro(7))`.

Saída esperada: 14

03 Soma — NÍVEL 1

Crie `somar(a, b)` que devolve a + b. Mostre `console.log(somar(3, 5))`.

Saída esperada: 8

04 Área do retângulo — NÍVEL 1

Crie `area(largura, altura)` e mostre `` `Área: ${area(5, 3)}` ``.

Saída esperada: Área: 15

05 É par? — NÍVEL 2

Crie `ehPar(n)` que devolve true ou false. Mostre `console.log(ehPar(4), ehPar(7))`.

Saída esperada: true false

06 Maior de dois — NÍVEL 2

Crie `maior(a, b)` que devolve o maior número. Mostre `console.log(maior(12, 30))`.

Saída esperada: 30

07 Conversor de temperatura — NÍVEL 2

Crie `celsiusParaFahrenheit(c)` que devolve `c * 9 / 5 + 32` e mostre a conversão de 25°C.

Saída esperada: 25°C = 77°F

08 Situação do aluno

NÍVEL 2

Crie `situacao(nota)` que devolve Aprovado (7+), Recuperação (5+) ou Reprovado. Mostre para 8, 6 e 3.

Saída esperada: Aprovado / Recuperação / Reprovado

09 Desconto com valor padrão

NÍVEL 3

Crie `aplicarDesconto(valor, porcentagem = 10)`. Mostre `aplicarDesconto(100)` e `aplicarDesconto(100, 25)`.

Saída esperada: 90 / 75

10 Arrow function

NÍVEL 3

Crie `const triplo = (n) => ...` e mostre `triplo(5)`.

Saída esperada: 15

11 Somar lista

NÍVEL 3

Crie `somarLista(lista)` que soma os itens com um laço. Mostre `somarLista([1, 2, 3, 4])`.

Saída esperada: 10

12 Contar vogais

DESAFIO

Crie `contarVogais(texto)` usando for...of e `"aeiou".includes(letra)`. Mostre `contarVogais("javascript")`.

Saída esperada: 3

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.